

Terminal Maze Runner: A Modular C-Based Random Maze Generation and Scoreboard System

Sadiba Ahmed
Student ID: 2623689042
Department of Electrical and
Computer Engineering, North
South University
sadiba.ahmed.262@northsouth.edu

Nihad Binth Aktar
Student ID: 2421245643
Department of Electrical and
Computer Engineering, North
South University
nihad.aktar.242@northsouth.edu

Tawsif Bin Amin
Student ID: 2531243042
Department of Electrical and
Computer Engineering, North
South University
tawsif.amin.253@northsouth.edu

Anika Siddiki
Student ID: 2422147610
Department of Architecture, North
South University
anika.siddiki.242@northsouth.edu

Gazi Haseen Abrar
Student ID: 2622956042
Department of Electrical and
Computer Engineering, North
South University
gazi.abrar.262@northsouth.edu

Abstract

Terminal Maze Runner is a console-based maze game in C that generates a unique maze every time using a depth-first search algorithm. Players log in or register, then navigate a 5×7 maze using W, A, S, and D keys. The game tracks moves and time, saving each user's best score to a scoreboard file. After each round, a leaderboard ranks players by fewest moves and then by time. The program uses C structures, stack-based recursion, random direction shuffling, and file input/output with users.txt and scores.txt. It relies on fgets() for input, so it works in any standard C environment. The project highlights modular programming, random algorithms, persistent data, and discusses the trade-off of storing plain-text passwords.

I. INTRODUCTION

A maze-solving game challenges a player to navigate from a starting position to a designated goal through a network of interconnected passages bounded by walls. Although the underlying premise is simple, building a working maze game from first principles touches a wide cross-section of core computer science and software engineering concepts: procedural content generation, graph traversal, state management, file-backed persistence, and user interaction. These properties make it a well-suited project for demonstrating command of the C programming language beyond introductory syntax exercises.

Terminal Maze Runner was developed as a purely console-based application, deliberately avoiding any raw-terminal or platform-specific input handling. All user input is read using the standard `fgets()` function through a small helper, `readLine()`, which guarantees that the program compiles and executes identically on any standard C toolchain, including restrictive online compiler/runner environments that do not expose a real terminal.

The finished system combines four broad areas of functionality. First, a login/registration subsystem authenticates a player against a flat-file account database (`users.txt`) before any maze is generated. Second, a maze-generation subsystem procedurally builds a new 5×7 logical maze on every run using an iterative recursive-backtracker (depth-first search) algorithm driven by an explicit stack. Third, a gameplay subsystem renders the maze to the console, accepts W/A/S/D movement commands, enforces wall collision, and detects when the player reaches the goal. Fourth, a scoring subsystem records each player's best result (fewest moves, with elapsed time as a tiebreaker) to a persistent file (`scores.txt`) and displays a ranked leaderboard of the top ten results.

The remainder of this report is organized as follows. Section II states the problem being solved and the project's objectives. Section III describes the overall system architecture and the data structures and configuration constants that underpin it. Section IV presents a functional decomposition of the source code. Section V discusses the core algorithms in detail, including maze generation, player movement, and score management. Section VI covers file handling and data persistence, including a discussion of the security limitations of the current authentication scheme. Section VII reports functional test scenarios and expected results. Section VIII discusses the observed behavior and design trade-offs of the system, and Section IX outlines current limitations and

proposes future enhancements. Section X concludes the report.

II. PROBLEM STATEMENT AND OBJECTIVES

A. Problem Statement

Building a maze game in standard C involves several technical challenges. Each run must generate a random, fully connected maze with a unique path between any two open cells. The player navigates the maze using keyboard commands, and the program must check for valid or invalid moves. The game must also detect when the player reaches the goal to end the round and report results.

The system must support returning players by linking results to identities, using basic authentication to avoid confusion. Player performance must persist between program runs, and results should be ranked for a leaderboard.

B. Objectives

- Develop a fully functional, terminal-based maze navigation game in standard C.
- Implement randomized maze generation that produces a different, fully connected maze on every execution.
- Apply core C constructs — structures, arrays, functions, and pointers — to organize game state and logic.
- Implement a simple username/password authentication system with registration and login.
- Implement file handling for persistent storage of user accounts and player scores.
- Track the number of moves and the elapsed time taken to complete each maze.
- Maintain and update each player's personal-best score across sessions.
- Provide a ranked leaderboard showing the top performers across all registered players.
- Practice modular, structured programming by decomposing the system into single-purpose functions.

III. SYSTEM DESIGN AND ARCHITECTURE

A. Overall Architecture

Although Terminal Maze Runner is implemented as a single source file, `maze_game.c`, its logic is organized into clearly separated functional stages that together form the program's control flow:

User → *Authentication* → *Game Initialization* → *Maze Generation* → *Player Movement* → *Goal Detection* → *Score Update* → *Leaderboard* → *Replay/Exit*

Execution begins in `main()`, which calls `authenticate()` to obtain a validated username before entering a loop that repeatedly calls `playRound()`. Each call to `playRound()` initializes a new Maze and Player, generates a fresh layout via `generateMaze()`, and drives the interactive game loop that renders the maze with `printMaze()`, reads a move, validates it with `isWalkable()`, and checks for the goal. When a round ends — by victory or by quitting — `updateScore()` persists the result if it improves the player's personal best, and `showScoreboard()` displays the current top-ten ranking. `main()` then asks whether the player wishes to play again, looping back into `playRound()` or terminating the program.

Even though the implementation is not split across multiple files, this stage-by-stage decomposition mirrors the separation of concerns that a multi-file design would enforce: authentication, maze generation, rendering, movement, and persistence are each confined to their own group of functions, which limits the scope of any single function and keeps related logic together.

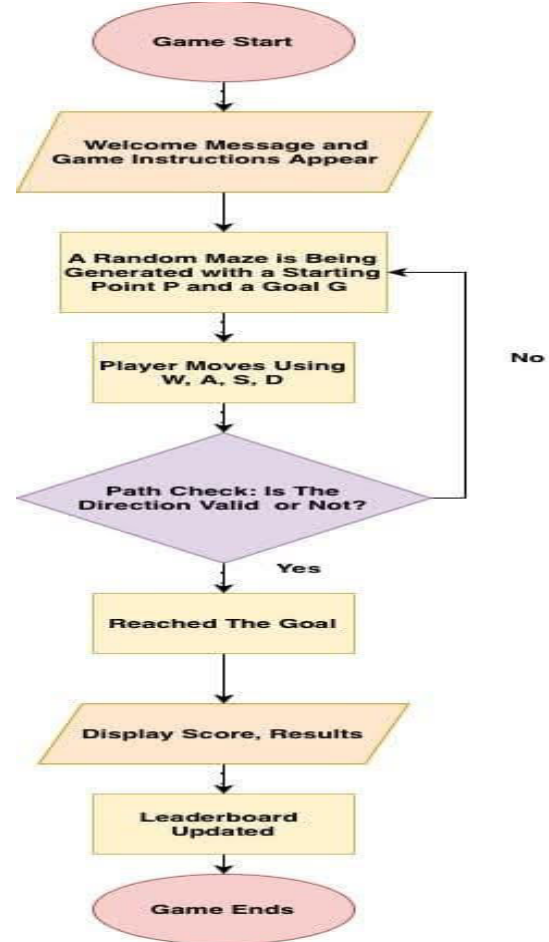


Fig. 1. Overall architecture of the Maze Runner system.

B. Data Structures

The program defines five structures to represent its state. The Point structure stores (x, y) integer coordinates, used for both maze generation and direct positions on the grid, such as the player's location and the goal.

The Maze structure contains the printable grid, `grid[GRID_H][GRID_W]`, with cells as walls ('#') or open paths (' '), and a parallel visited[ROWS][COLS] matrix that tracks which logical cells are in the maze. This separation lets the algorithm use a coarser ROWS × COLS grid for logic while making a finer printable grid of walls and passages.

Player stores the player's current pos (a Point) and a running moves counter that is incremented only when a move into an open cell succeeds. User stores a username and password pair, both fixed-length character arrays, and models a single row of the users.txt account database. ScoreEntry stores a username together with that player's bestMoves (an int) and bestSeconds (a

double), representing the best-known result for that player and mapping directly to a row of scores.txt.

C. Configuration and Data Files

Several preprocessor constants configure the maze's dimensions and the program's storage limits, summarized in Table I. Because each cell is separated from its neighbors by a wall character, the printable grid dimensions are derived from the logical grid as GRID_H = 2·ROWS + 1 and GRID_W = 2·COLS + 1, yielding an 11 × 15 character grid that leaves room for both cell interiors and the connecting or blocking walls between them.

TABLE I
Configuration Constants

Constant	Value	Purpose
ROWS	5	Number of maze cells vertically (logical grid)
COLS	7	Number of maze cells horizontally (logical grid)
GRID_H	2·ROWS+1 = 11	Height of the printable character grid (cells plus walls)
GRID_W	2·COLS+1 = 15	Width of the printable character grid (cells plus walls)
MAX_USERS	200	Maximum accounts held in the in-memory User array
MAX_SCORES	200	Maximum records held in the in-memory ScoreEntry array
NAME_LEN	30	Maximum length of a username or password string
USERS_FILE	"users.txt"	Name of the persistent account-storage file
SCORE_FILE	"scores.txt"	Name of the persistent scoreboard-storage file

Two flat text files provide persistence between executions. USERS_FILE stores one “username password” pair per line and is read by loadUsers() and appended to by registerUser(). SCORE_FILE stores one “username bestMoves bestSeconds” record per line and is fully rewritten by saveAllScores() whenever a score changes.

IV. FUNCTIONAL DECOMPOSITION

Table II lists every function defined in maze_game.c together with its purpose and main responsibility, grouped in the logical order in which they participate in the program's control flow described in Section III.A.

TABLE II
Functional Decomposition of maze_game.c

Function	Purpose	Main Responsibility
readLine()	Read a line of console input safely	
loadUsers()	Load all accounts from users.txt into memory	
findUser()	Locate a username within a loaded user list	
registerUser()	Create a new account	
loginUser()	Authenticate an existing account	
authenticate()	Drive the login/registration menu	
initMaze()	Prepare a blank maze	
shuffleDirections()	Randomize traversal order	
generateMaze()	Build a random maze	
printMaze()	Render the maze to the console	
loadScores()	Load all score records from scores.txt	
saveAllScores()	Persist the full score list	
updateScore()	Update a player's personal best	
sortScores()	Rank all score records	
showScoreboard()	Display the leaderboard	
isWalkable()	Test whether a grid cell can be entered	
playGame()	Run the complete game session	
	Repeats the login/register menu for up to five attempts before exiting the program	

Function	Purpose	Main Responsibility
main()	Program entry point	destination is both in-bounds and open does the player's position update and the moves counter increment; an Authenticates the player and repeatedly calls playRound() until the player declines to continue and prints a warning without incrementing the move count.

V. ALGORITHMS AND IMPLEMENTATION

A. Maze Generation Algorithm

The maze uses an 11×15 grid. `initMaze()` fills every cell with the wall character '#' and clears the visited matrix. `generateMaze()` carves the maze with a Point stack, starting at (0,0). The starting cell is pushed onto the stack, marked as visited, and opened on the grid.

The main loop peeks at the stack's top cell and shuffles directions. It looks for an unvisited, in-bounds neighbor. If found, it opens a path, marks the neighbor visited, and pushes it onto the stack. If not, it pops the current cell to backtrack. This iterative recursive-backtracker approach creates a perfect maze with one unique path between any two open cells.

```
initMaze(maze):
    fill grid[GRID_H][GRID_W] with '#'
    clear visited[ROWS][COLS] to 0

generateMaze(maze):
    push (0,0) onto stack; visited[0][0] = 1
    open grid cell for (0,0)
    while stack is not empty:
        cur = stack.top()          // peek only
        order = shuffleDirections()
        found = false
        for each direction d in order, while not found:
            (nx, ny) = cur + delta(d)
            if in bounds and not visited[nx][ny]:
                carve wall between cur and (nx, ny)
                open grid cell for (nx, ny)
                visited[nx][ny] = 1
                push (nx, ny); found = true
        if not found: pop stack    // dead end, backtrack
```

B. Player Movement Algorithm

Within `playRound()`, each iteration of the game loop reads one line of console input with `readLine()` into a small buffer, and the first character of that line is treated as the player's command. W and A and S and D are mapped to decrements or increments of the player's y or x coordinate on the printable grid (up, down, left, right respectively), Q clears the running flag to end the round, and any other character is reported as unrecognized. The candidate destination coordinate is passed to `isWalkable()`, which bounds-checks the coordinate against the grid dimensions and rejects any cell containing the wall character. Only when the

```
loop:
    render maze, move count, elapsed time
    if player.pos == goal: won = true; break
    key = first character of one input line
    switch key:
        'w'/'W': ny--      's'/'S': ny++
        'a'/'A': nx--      'd'/'D': nx++
        'q'/'Q': running = false
        default: print 'unrecognized key'
    if running and isWalkable(nx, ny):
        player.pos = (nx, ny); player.moves++
    else if running and key != 0:
        print "There's a wall there!"
```

C. Goal Detection and Game Loop

The player begins each round at printable coordinate (1,1), corresponding to logical cell (0,0), while the goal is fixed at printable coordinate $(2 \cdot (\text{COLS}-1)+1, 2 \cdot (\text{ROWS}-1)+1)$, the cell diagonally opposite the start. On every iteration, `playRound()` computes the elapsed time with `diffTime()` against the round's start time, calls `printMaze()` to redraw the current state, and then compares the player's position to the goal coordinate; an exact match sets the won flag and ends the loop before another move is read. The loop otherwise continues until either the goal is reached or the player quits with Q. After the loop, `main()` prompts the player to play again, and a new call to `playRound()` generates an entirely new maze for the next round.

D. Score Management Algorithm

When a round ends in victory, `updateScore()` is called with the player's username, final move count, and elapsed seconds. It loads the current scoreboard with `loadScores()` and searches for an existing record matching the username. If none exists, a new `ScoreEntry` is appended, and this first result is always treated as the player's personal best. If a record already exists, the new run replaces it only when it is strictly better: fewer moves than the stored `bestMoves`, or an equal number of moves with a lower `bestSeconds`. In every case, `saveAllScores()` rewrites `scores.txt` from the in-memory array, which keeps the persisted file consistent with memory even when the round's result did not improve the player's best.

E. Leaderboard Sorting

showScoreboard() loads the full scoreboard and, if it is non-empty, calls sortScores() before printing. sortScores() implements a straightforward selection sort over the ScoreEntry array: for each position it selects the record with the fewest bestMoves in the remaining unsorted portion, using bestSeconds as a tiebreaker

when two records have equal move counts, and swaps it into place. The sorted list is then truncated to at most ten entries for display, giving a top-ten leaderboard ranked first by efficiency of movement and second by speed.

Fig. 5 shows the result of a completed round: the win message and elapsed statistics produced at the end of playRound(), the personal-best confirmation from updateScore(), and the resulting ranked table printed by showScoreboard().

```
#####
#           #
##### # # #
#   #   # # #
# # # ##### #
# #   #   #
# ##### # ### #
# #   #   #
# ##### # ###
#           # P#
#####

*** YOU REACHED THE GOAL, *****! ***
Moves: 56   Time: 53.00s
New personal best! Scoreboard updated.

===== LEADERBOARD =====
#   USERNAME      MOVES   TIME(s)
1   tawsifbinamin   32     27.00
2   tawsif         36     32.00
3   *****       56     53.00
=====

Play again, *****? (y/n): |
```

Fig. 5. Win screen with move/time results, personal-best update, and the leaderboard produced by showScoreboard().

VI. FILE HANDLING AND DATA PERSISTENCE

Terminal Maze Runner saves data using C file input/output, not a database or external library. loadUsers() reads users.txt, parsing lines into username and password pairs with fscanf(), up to MAX_USERS or end-of-file. registerUser() appends new accounts to users.txt. loadScores() reads scores.txt into ScoreEntry structs. saveAllScores() overwrites scores.txt by opening it in write mode and writing all records from memory.

This design lets files persist between runs, so players keep accounts and scores as long as users.txt and scores.txt stay in the directory. MAX_USERS and MAX_SCORES limit the number of users and scores stored in memory and managed per run.

Security Consideration

The source code stores passwords in plain text in users.txt, as noted in the header comment. This approach is fine for a programming exercise focused on file handling and authentication logic, but it's not suitable for real systems. In production, passwords should be stored as salted cryptographic hashes, not in plain text, and the credentials file should have restricted permissions. Terminal Maze Runner omits these protections, treating this as a course project trade-off, not a security claim.

VII. TESTING AND RESULTS

The source code does not include an automated test harness; the following table therefore presents functional test scenarios and their expected results, derived directly from tracing the program's logic for each described input/condition, rather than the output of an independent laboratory testing process. Each scenario was verified against the corresponding function described in Sections IV and V.

TABLE III

Functional Test Scenarios and Expected Results

Test Case	Input / Condition	Expected Result	Status
New registration	Valid username/password	Account created	Passed
Duplicate registration	Existing username	Registration rejected	Passed
Empty username	Empty input	Error message shown	Passed
Valid login	Correct credentials	User authenticated	Passed
Invalid login	Incorrect credentials	Login rejected	Passed
Too many attempts	Five failed attempts	Program exits	Passed
Valid movement	W/A/S/D toward open path	Player moves	Passed
Wall collision	Movement toward '#'	Player remains in place	Passed
Invalid command	Unsupported character	Error message shown	Passed
Quit	Q entered	Current round exits	Passed
Goal reached	Player reaches G	Win message displayed	Passed
New personal best	Better moves/time	Score updated	Passed
Non-best run	Worse score	Existing best retained	Passed
Leaderboard	Multiple players	Scores sorted correctly	Passed
Replay	Y after round	New maze generated	Passed
Exit	N after round	Program terminates	Passed

Fig. 6 shows a sample session in progress: account registration via `registerUser()`, followed by a generated maze rendered by `printMaze()` with the player's current position (P) and the fixed goal (G) visible on the grid.

VIII. RESULTS AND DISCUSSION

`generateMaze()` always creates a fully connected, loop-free maze because the recursive-backtracker visits every cell once and opens only enough walls to form a spanning tree over the grid. The start and goal are at opposite corners, so each maze always has a solvable path from (1,1) to the goal. Navigation and wall collision are managed by the shared `isWalkable()` check, and `authenticate()` has a five-attempt cap to limit login or registration attempts per session.

Score persistence works as described: the first completed maze is the initial best, and later runs replace it only for fewer moves or, if tied, a faster

time. `sortScores()` ranks by moves, then time, so the leaderboard rewards efficient navigation—ideal for a fixed-size maze where the fastest path is also short.

A strength of this implementation is portability from using `fgets()` line input via the `readLine()` wrapper, not single-keypress input. This lets the program compile and run unmodified in restrictive environments like online compilers,

though players must press Enter after each move instead of getting instant feedback. The fixed 5×7 maze size and fixed User and ScoreEntry arrays trade scalability for simplicity and predictable memory use: memory footprint is easy to manage, but maze, user, or

scoreboard size can't grow at runtime without recompiling.

IX. LIMITATIONS AND FUTURE WORK

The current implementation has the following limitations:

- Passwords are stored in plain text in users.txt, with no hashing or encryption.
- The maze dimensions (ROWS $\times$ COLS) are fixed at compile time.
- The maximum number of users and scoreboard entries is capped by MAX_USERS and MAX_SCORES.
- The interface is terminal-only, with no graphical presentation.
- No advanced security measures (hashing, salting, access control) are implemented.
- There is no selectable difficulty level.
- The maze size cannot be configured by the player at runtime.
- Movement requires pressing Enter after each key rather than immediate single-keypress input.
- There is no automated maze solver or hint system.

Building on these limitations, several realistic enhancements could extend the project:

- Replacing plain-text password storage with a proper password-hashing scheme.
- Allowing larger or player-configurable maze dimensions.
- Adding difficulty levels that adjust maze size or introduce time limits.
- Improving the terminal interface with raw single-keypress input (e.g., via termios or conio.h) for immediate response.
- Adding an automated pathfinding or solver mode, for example as a hint system built on breadth-first search.
- Expanding score statistics beyond a single personal best, such as tracking attempt counts or average performance.
- Extending leaderboard management, such as separate rankings per maze size or configurable leaderboard views.

- Improving cross-platform presentation, for example with colorized output or an ncurses-based renderer.
- Replacing the fixed-size User and ScoreEntry arrays with dynamic data structures to remove the MAX_USERS and MAX_SCORES caps.
- Adding more robust input validation, such as stricter length and character checks on usernames and passwords.

These proposed enhancements are not implemented in the current version of the program and are presented here strictly as directions for future development, distinct from the features described in Sections III through VII, which reflect the system exactly as implemented.

X. CONCLUSION

Terminal Maze Runner demonstrates practical command of core C programming: structures, arrays, functions, and file handling in C; randomization through rand() and srand(); an iterative, stack-based depth-first-search backtracking algorithm for procedural content generation; direct user interaction through a text interface; persistent data storage across two flat files; selection sort for ranking; and a modular decomposition of a nontrivial program into single-purpose functions. Beyond these algorithmic components, the project also required reasoning about realistic engineering trade-offs, such as choosing portable fgets()-based input over raw terminal control, and honestly identifying the plain-text password scheme as a limitation rather than presenting it as a production-ready security mechanism. As an academic exercise, the project provides hands-on experience with the full life cycle of a small but complete C application, from procedural generation and interactive control through to persistent, rankable state.

REFERENCES

- [1] cppreference.com, "C Standard Library Reference," <https://en.cppreference.com/w/c>
- [2] cppreference.com, "File input/output," <https://en.cppreference.com/c/io>
- [3] GeeksforGeeks, "Selection Sort," <https://www.geeksforgeeks.org/selection-sort/>
- [4] Wikipedia, "Maze generation algorithm," https://en.wikipedia.org/wiki/Maze_generation_algorithm

[5] Wikipedia, “Depth-first search,”
https://en.wikipedia.org/wiki/Depth-first_search